

Trabajo fin de grado

Desarrollo de un Tutor Socrático basado en LLMs para la enseñanza de Programación



Ignacio García Hernanz

Escuela Politécnica Superior
Universidad Autónoma de Madrid
C/ Francisco Tomás y Valiente nº 11

UNIVERSIDAD AUTÓNOMA DE MADRID
ESCUELA POLITÉCNICA SUPERIOR



Grado en Ingeniería Informática

TRABAJO FIN DE GRADO

**Desarrollo de un Tutor Socrático basado en LLMs
para la enseñanza de Programación**

SocratiCode

Autor: Ignacio García Hernanz
Tutor: Oscar Delgado Ben Mohatar

mayo 2026

Todos los derechos reservados.

Queda prohibida, salvo excepción prevista en la Ley, cualquier forma de reproducción, distribución comunicación pública y transformación de esta obra sin contar con la autorización de los titulares de la propiedad intelectual.

La infracción de los derechos mencionados puede ser constitutiva de delito contra la propiedad intelectual (*arts. 270 y sgts. del Código Penal*).

DERECHOS RESERVADOS

© Fecha de entrega por UNIVERSIDAD AUTÓNOMA DE MADRID
Francisco Tomás y Valiente, n.º 1
Madrid, 28049
Spain

Ignacio García Hernanz

Desarrollo de un Tutor Socrático basado en LLMs para la enseñanza de Programación

Ignacio García Hernanz

C\ Francisco Tomás y Valiente N.º 11

IMPRESO EN ESPAÑA – PRINTED IN SPAIN

Dedicatória...

Cita famosa...

Autor

PREFACIO

Este estilo de $\text{\LaTeX} 2_{\epsilon}$ ha sido diseñado con dos propósitos. El primer propósito es el de facilitar en lo posible la escritura de trabajos de fin de grado y de máster y de tesis doctorales. En ese sentido se han diseñado un conjunto de comandos que simplifican la escritura y diseño de estos trabajos pero que reducen en cierta forma las capacidades de los paquetes de $\text{\LaTeX}$ utilizados. Sin embargo, dado que los paquetes están incluidos en esta clase, pueden utilizarse directamente y hacer diseños más complejos pero si se hace esto se recomienda mantener una estética coherente con el resto del documento.

El segundo de los propósitos es que estos documentos mantengan una estética uniforme en la Universidad Autónoma de Madrid y fomentar una imagen corporativa en documentos tan relevantes como los trabajos de fin de grado o de máster y las tesis doctorales. Por ese motivo se recomienda mantener una coherencia estética en todo momento. El diseño facilita esa coherencia pero es posible salirse del diseño si se mantiene dicha coherencia.

Como creador de este estilo espero fervientemente que al usar este estilo te sientas cómodo y te facilite la escritura de un documento que es muy relevante en esta etapa de tu vida. Para facilitártela aún más, el código fuente de este documento también está disponible en tu ordenador o en overleaf para que te sirva a modo de ejemplo.

Eloy Anguiano Rey

AGRADECIMIENTOS

En primer lugar me gustaría agradecer a la Escuela Politécnica Superior por su apoyo para la creación de esta clase y que sea el formato básico para la creación de tesis, trabajos fin de grado y trabajos fin de master.

En particular quiero destacar el trabajo realizado por Fernando López-Colino por su apoyo en la comisión de imagen institucional y por sus comentarios para mejorar este estilo.

También quiero tener un recuerdo para Carmen Navarrete Navarrete dado que este estilo comencé a crearlo a partir de sus necesidades a la hora de escribir la tesis. Y por supuesto a no quiero olvidarme de mi esposa e hijos que han servido de conejillos de indias en sus correspondientes trabajos fin de master y de grado. No quiero olvidar a todos los estudiantes que me pidieron este estilo y lo han usado para presentar sus trabajos pero son muchos y podría olvidarme de alguno, por tanto, mi agradecimiento en general a todos ellos.

RESUMEN

En nuestra Escuela se producen un número considerable de documentos, tantos docentes como investigadores. Nuestros alumnos también contribuyen a esta producción a través de sus trabajos de fin de grado, máster y tesis. El objetivo de este material es facilitar la edición de todos estos documentos y a la vez fomentar nuestra imagen corporativa, facilitando la visibilidad y el reconocimiento de nuestro Centro.

En este sentido se ha intentado diseñar un estilo de $\text{\LaTeX} 2_{\epsilon}$ que mantenga una imagen corporativa y con comandos simples que permitan mantener la imagen corporativa con la calidad necesaria sin olvidar las necesidades del autor. Para ello se han creado un conjunto de comandos simples en torno a paquetes complejos. Estos comandos permiten realizar la mayoría de las operaciones que un documento de este tipo pueda necesitar.

Así mismo se puede controlar un poco el diseño del documento a través de las opciones del estilo pero siempre manteniendo la imagen institucional.

PALABRAS CLAVE

Diseño de documento, $\text{\LaTeX} 2_{\epsilon}$, thesis, trabajo fin de grado, trabajo fin de master

ABSTRACT

In our School a considerable number of documents are produced, as many aducational as research. Our students also contribute to this production through his final degree, master and thesis projects. The objective of this material is to facilitate the editing of all these documents and at the same time to promote our corporate image, facilitating the visibility and recognition of our center.

In this sense we have tried to design a style of $\text{\LaTeX}2_{\epsilon}$ that maintains a corporate image and with simple commands that allow to maintain the corporate image with the necessary quality without forgetting the needs of the author. For this, a set of simple commands have been created around complex packages. These commands allow you to perform most of the operations that a document of this type may need.

Likewise, you can control a little the design of the document through the options of the style but always maintaining the institutional image.

KEYWORDS

Document design, $\text{\LaTeX}2_{\epsilon}$, thesis, final degree project, final master project

ÍNDICE

1	Introducción	1
1.1	Motivación	1
1.2	Objetivos	2
1.3	Estructura del Proyecto	2
2	Estado del arte	5
2.1	Fundamentos Pedagógicos en la Enseñanza de la Programación	5
2.1.1	Dificultades en el aprendizaje inicial (CS1)	5
2.1.2	El Método Socrático como mitigación	6
2.2	Inteligencia Artificial Generativa en la Educación	6
2.2.1	Riesgos del facilismo algorítmico	6
2.2.2	Sistemas de tutoría guiada (Casos de Éxito)	7
2.2.3	Ingeniería de Prompts en educación	9
2.3	Modelos de Lenguaje Extenso (LLMs) y Despliegue Local	10
2.3.1	APIs en la nube vs. Inferencia Local	10
2.3.2	Ecosistema y clasificación de modelos de lenguaje de código abierto	11
2.3.3	Optimización de modelos ligeros	13
2.4	Seguridad y aislamiento en la ejecución de código	13
2.5	Arquitecturas web para entornos interactivos	14
2.5.1	Paradigma de Aplicación de Página Única (SPA)	14
2.5.2	Protocolos de comunicación en tiempo real	15
2.6	Conclusiones del Estado del Arte	15
3	Análisis y diseño	17
3.1	Metodología	17
3.2	Descripción del sistema - SocratiCode	17
3.3	Requisitos funcionales y no funcionales	17
3.3.1	Requisitos funcionales	17
3.3.2	Requisitos no funcionales	17
3.4	Casos de uso	17
3.5	Diagrama de modelo de datos (Clases/Entidad-Relación)	17
3.6	Diagrama de componentes y arquitectura	17
3.6.1	Diagrama de arquitectura	17

3.6.2 Diagramas de secuencia de las funcionalidades principales	17
3.7 Conclusiones	17
4 Desarrollo e implementación	19
4.1 Entorno de programación	19
4.2 Tecnologías y librerías	19
4.2.1 Framework Backend (Django / Python)	19
4.2.2 Framework Frontend (Vue 3 / Vite)	19
4.2.3 Motor de Inteligencia Artificial (Ollama / Llama 3.2)	19
4.2.4 Motor de compilación y ejecución (Piston / Docker)	19
4.2.5 Control de versiones y despliegue (Git / GitHub)	19
4.3 Implementación	19
4.3.1 Gestión de la lógica socrática (Prompt Engineering)	19
4.3.2 Implementación del flujo de datos en tiempo real (SSE)	19
4.3.3 Integración y aislamiento del motor de ejecución (Docker/Piston)	19
4.4 Interfaz	19
4.4.1 Navegación y vista principal del tutor	19
4.4.2 Personalización y editor de código	20
4.5 Verificación y Pruebas del sistema	20
4.6 Conclusiones	20
5 Pruebas	21
5.1 Pruebas de seguridad e integridad (Pentesting del Sandbox)	21
5.2 Encuesta de experiencia de usuario y validación pedagógica	21
6 Conclusiones y trabajo futuro	23
6.1 Conclusiones	23
6.2 Trabajo futuro	23
Bibliografía	27
Apéndices	29
A Sprints desarrollados	31
B Diagramas y casos de uso	33
C Navegación e interfaces extra	35
D Resultados de encuesta y auditoría de seguridad	37

LISTAS

Lista de algoritmos

Lista de códigos

Lista de cuadros

Lista de ecuaciones

Lista de figuras

2.1	Arquitectura del sistema CS50	7
2.2	Arquitectura del sistema CodeAid	8
2.3	Arquitectura del sistema TreeInstruct	9

Lista de tablas

2.1	Comparativa técnica de LLMs en Ollama (Auditoría Final)	12
-----	---	----

Lista de cuadros

INTRODUCCIÓN

El presente capítulo sirve como punto de partida y ofrece una panorámica global del Trabajo Fin de Grado. Su propósito es contextualizar el desarrollo de SocratiCode y establecer las bases sobre las que se fundamenta el proyecto.

El recorrido comienza en la Sección 1.1, donde se analizan las diferentes razones que han impulsado su realización. En ella se destaca la creciente necesidad de apoyar la enseñanza de la programación haciendo uso de capacidades basadas en Inteligencia Artificial y aplicando metodologías de aprendizaje activo como el diálogo socrático. A continuación, la Sección 1.2 define de manera concreta las metas a conseguir con este sistema, abarcando tanto las exigencias desde el punto de vista del desarrollo técnico como el impacto educativo que se espera lograr. Para concluir, la Sección 1.3 proporciona un breve mapa de lectura de la presente memoria, delineando la organización y la temática que el lector encontrará a lo largo de los capítulos venideros.

1.1. Motivación

En los últimos años, la Inteligencia Artificial ha evolucionado hasta alcanzar la capacidad de generar, testear y ejecutar código con una competencia equiparable a la de un ingeniero cualificado. No obstante, continúa siendo una herramienta que requiere de un usuario capaz de guiarla y supervisar sus resultados. Un estudiante que se inicia en la programación carece de dichas facultades analíticas, sin embargo, tiene a su disposición una tecnología capaz de resolver, en cuestión de segundos, los retos iniciales inherentes al aprendizaje del desarrollo de software.

La prohibición del uso de estas herramientas en el ámbito académico puede resultar contraproducente, puesto que el aprendizaje de la programación puede generar altos niveles de frustración en principiantes. Esta situación conlleva el riesgo de que el alumnado abandone el estudio o recurra a soluciones automatizadas que impidan el desarrollo del pensamiento computacional, facultad indispensable para establecer los fundamentos técnicos que posteriormente, permitirán aprovechar todo el potencial de estas tecnologías avanzadas de manera eficaz.

Ante este nuevo paradigma, el enfoque del programador profesional no debe residir exclusivamente

en la sintaxis de los diversos lenguajes de programación. Por el contrario, el valor diferencial se desplaza hacia el diseño de sistemas, la descomposición modular en subtarefas para su implementación y la selección estratégica de las herramientas y tecnologías más adecuadas para cada requerimiento técnico.

Bajo este contexto, **SocratiCode** nace como una respuesta técnica y pedagógica a la actual dicotomía entre la prohibición del uso de la IA y la automatización excesiva en el aprendizaje. El proyecto propone una arquitectura basada en modelos de lenguaje locales que, en lugar de generar soluciones directas, actúan como tutores socráticos. Al integrar un entorno de ejecución seguro con una asistencia iterativa, la plataforma busca transformar la IA en un catalizador del pensamiento computacional, garantizando que el alumno mantenga el control del flujo lógico y el esfuerzo cognitivo en su aprendizaje.

1.2. Objetivos

El objetivo principal de este Trabajo Fin de Grado es diseñar, implementar y validar una plataforma educativa de programación que integre un tutor basado en inteligencia artificial local con una metodología socrática de enseñanza y un entorno de ejecución de código aislado y seguro. Para alcanzar dicho objetivo, se plantean los siguientes objetivos específicos:

- Investigar el estado del arte en sistemas de tutoría inteligente y plataformas de aprendizaje de programación, analizando las capacidades actuales de la IA generativa para fomentar el pensamiento computacional.
- Evaluar y seleccionar las soluciones tecnológicas más eficientes para la inferencia local de modelos de lenguaje (Ollama) y la orquestación de procesos aislados, garantizando un compromiso óptimo entre privacidad, rendimiento y seguridad.
- Diseñar una arquitectura de sistema integral que armonice el procesamiento de lenguaje natural, un motor de ejecución de código efímero y una interfaz de usuario reactiva orientada a la experiencia pedagógica.
- Adoptar una metodología de desarrollo iterativa basada en principios ágiles, permitiendo la evolución progresiva del sistema desde un Producto Mínimo Viable (MVP) hasta una plataforma robusta mediante ciclos de refinamiento continuo.
- Elaborar la documentación técnica detallada siguiendo estándares de ingeniería del software, abarcando desde la captura de requisitos y casos de uso hasta el modelado de la arquitectura y diagramas estructurales.
- Validar la plataforma mediante auditorías de seguridad sobre el entorno de ejecución y la realización de cuestionarios de experiencia de usuario para contrastar la efectividad real del tutor socrático.

1.3. Estructura del Proyecto

La presente memoria se organiza en seis capítulos cuyo contenido se describe a continuación:

El Capítulo 1 introduce el contexto y la motivación que justifican la realización del proyecto, expone

los principales objetivos técnicos y pedagógicos planteados, y proporciona una visión general de la estructura del documento.

El Capítulo 2 recoge el estado del arte, abordando los fundamentos pedagógicos del método socrático, el impacto de la Inteligencia Artificial generativa en la educación y las ventajas de los modelos de lenguaje de ejecución local. Asimismo, se analizan las tecnologías de aislamiento de procesos (sandboxing) y las arquitecturas web necesarias para sistemas educativos interactivos en tiempo real.

El Capítulo 3 presenta el análisis y diseño del sistema, describiendo la metodología de trabajo seguida, los requisitos funcionales y no funcionales, los casos de uso, el modelo de datos y la arquitectura de componentes.

El Capítulo 4 detalla el proceso de desarrollo e implementación, describiendo el entorno de trabajo, las tecnologías empleadas y las decisiones de diseño más relevantes adoptadas durante la construcción del sistema.

El Capítulo 5 recoge las pruebas realizadas sobre el sistema, tanto las pruebas de seguridad del sandbox como la validación pedagógica con usuarios finales.

El Capítulo 6 expone las conclusiones del proyecto y propone líneas de trabajo futuro.

ESTADO DEL ARTE

En este capítulo se analiza el contexto técnico y educativo en el que se enmarca **SocratiCode**. En primer lugar, se examinan los fundamentos pedagógicos que justifican la necesidad de nuevas herramientas en la enseñanza de la informática. Posteriormente, se realiza un recorrido por las soluciones tecnológicas actuales y los casos de éxito que han integrado la Inteligencia Artificial en el aula.

2.1. Fundamentos Pedagógicos en la Enseñanza de la Programación

Para entender el propósito de **SocratiCode**, es necesario analizar primero la realidad de las aulas de informática. El desarrollo de una plataforma de este tipo no responde solo a un interés tecnológico, sino a la búsqueda de una solución para dos problemas críticos: las barreras cognitivas que enfrentan los estudiantes principiantes y la imposibilidad de ofrecer una tutoría personalizada en entornos masificados. A continuación, se detallan estos desafíos y la propuesta del método socrático como vía para superarlos.

2.1.1. Dificultades en el aprendizaje inicial (CS1)

La enseñanza de la programación en los cursos introductorios, comúnmente denominados CS1 (*Computer Science 1*), representa uno de los desafíos pedagógicos más críticos en la educación superior contemporánea [1]. Históricamente, los programas académicos de informática experimentan altas tasas de abandono en etapas iniciales, con porcentajes de deserción que frecuentemente oscilan entre el 30 % y el 40 %. Esta problemática no responde únicamente a una falta de interés, sino a la complejidad de asimilar simultáneamente tanto las reglas gramaticales del código como la lógica necesaria para resolver problemas.

Para un estudiante que se enfrenta por primera vez a la programación, el entorno de desarrollo puede resultar abrumador. La combinación de conceptos abstractos y la rigidez de las herramientas genera una frustración profunda. Al no contar todavía con un modelo mental sólido, los alumnos tienen

grandes dificultades para entender por qué su código no funciona, sintiéndose a menudo incapaces de avanzar de forma autónoma. Idealmente, un profesor debería estar disponible para ofrecer pistas en el momento justo, pero en las facultades actuales, con clases masificadas, es físicamente imposible atender a cada estudiante de forma individualizada. Esta limitación es la que hace necesario buscar sistemas de tutoría automatizados que ofrezcan ese apoyo constante en tiempo real.

2.1.2. El Método Socrático como mitigación

Para resolver esta falta de acompañamiento sin caer en el extremo de resolverle el trabajo al alumno, la pedagogía moderna propone estrategias de instrucción guiada, destacando entre ellas el método socrático [2]. Este enfoque no busca dar respuestas, sino establecer un diálogo donde el tutor guía al estudiante mediante preguntas pequeñas y estratégicas. El objetivo es que el alumno “tire del hilo” de su propio razonamiento hasta descubrir, por sí mismo, dónde está el fallo en su lógica [3]. Al obligar al estudiante a verbalizar y reflexionar sobre lo que está intentando hacer, se consigue un aprendizaje mucho más profundo que el que se obtiene con una corrección automática [4].

Este apoyo, conocido como «andamiaje» (*scaffolding*), permite que el alumno no se rinda ante el primer bloqueo, ya que actúa como un soporte temporal que se adapta a su nivel para facilitar la transición hacia la autonomía en la resolución de problemas [5]. Este marco pedagógico resulta fundamental para el desarrollo de sistemas de tutoría inteligente que buscan transformar la interacción técnica en un proceso de descubrimiento guiado [2].

2.2. Inteligencia Artificial Generativa en la Educación

La llegada de la IA generativa ha cambiado por completo la forma de enseñar, representando una oportunidad enorme para personalizar el aprendizaje gracias a la capacidad de los modelos de lenguaje (LLM) para entender y corregir código. No obstante, para que esta tecnología funcione de manera óptima en el aula, es necesario modificar su comportamiento nativo utilizando una cuidada ingeniería de *prompts* junto con conceptos como el «andamiaje» para que el sistema se comporte como un tutor capaz de guiar al alumno mediante un diálogo estructurado [2,6]. El reto ahora es diseñar herramientas que ayuden a aprender sin convertirse en un atajo que evite el esfuerzo de pensar.

2.2.1. Riesgos del facilismo algorítmico

El uso masivo de herramientas como ChatGPT o GitHub Copilot ha convertido la resolución de problemas de programación en un proceso prácticamente instantáneo. Sin embargo, cuando estas tecnologías se emplean sin un marco pedagógico definido, surge el fenómeno del «facilismo algorítmico», donde el alumnado tiende a delegar la resolución lógica completa en la IA, limitándose a un

proceso de copia y pega que anula el esfuerzo cognitivo esencial para el aprendizaje [7]. Esta práctica fomenta una «ilusión de competencia», en la cual el estudiante posee un código funcional que es incapaz de explicar o depurar por sí mismo [8].

Lejos de democratizar el conocimiento, investigaciones recientes advierten que esta tendencia puede acabar ampliando la brecha entre estudiantes según su nivel previo. Mientras el experto gana productividad, el novato a menudo se siente frustrado al no comprender la lógica que la IA genera por él, lo que estanca su aprendizaje autónomo y convierte la herramienta en una barrera para el desarrollo del pensamiento computacional [8].

2.2.2. Sistemas de tutoría guiada (Casos de Éxito)

Para evitar estos problemas sin renunciar a la IA, diversas instituciones han desarrollado sistemas que siguen reglas pedagógicas muy estrictas. A continuación, se analizan tres de las implementaciones más relevantes y actuales en el ámbito académico.

CS50 Duck (Universidad de Harvard)

El referente más consolidado es el CS50 Duck, un asistente integrado en el entorno de desarrollo CS50.dev, una versión de Visual Studio Code adaptada para la nube mediante GitHub Codespaces. Utiliza una arquitectura RAG (*Retrieval-Augmented Generation*), descrita en la figura 2.1, para conectar las dudas del alumno con el material oficial del curso y filtros de seguridad que bloquean intentos de inyección de prompts. Toda su lógica operativa se gestiona desde un backend centralizado que utiliza archivos de configuración YAML para ajustar dinámicamente el comportamiento de GPT-4, asegurando que el modelo nunca entregue soluciones directas, sino pistas y preguntas de seguimiento que fomenten el razonamiento independiente y la técnica del *rubberducking* [9].

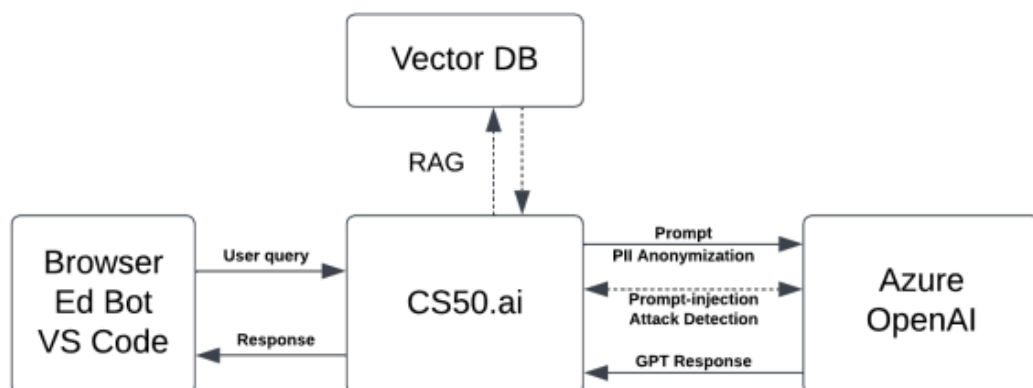


Figura 2.1: Arquitectura del sistema CS50.ai, detallando el flujo de consultas y el uso de técnicas RAG [10].

Para gestionar los costes de la API comercial, Harvard implementó un sistema de “corazones” que limita las consultas de los estudiantes, obteniendo un doble beneficio, por un lado obliga a los estudiantes a reflexionar sobre su pregunta antes de enviarla y por otro evita un elevado coste operativo [10].

CodeAid

El asistente CodeAid propone un modelo de ayuda estructurada basado en herramientas específicas como la explicación de conceptos técnicos o la resolución de dudas teóricas sin ejecución directa de código. Su arquitectura se apoya en una técnica de *few-shot prompting*, donde el modelo es entrenado con ejemplos de interacciones pedagógicas para mantener un tono de apoyo constante [11]. Uno de sus puntos fuertes es que, al seleccionar la opción de revisión de código, la IA sigue el flujo descrito por la figura 2.2 para garantizar la precisión de sus respuestas. En este proceso se genera internamente la solución correcta, pero el sistema solo muestra al estudiante un pseudocódigo que orienta su lógica sin regalarle la implementación final, obligándole a escribir cada línea en su propio editor [11].

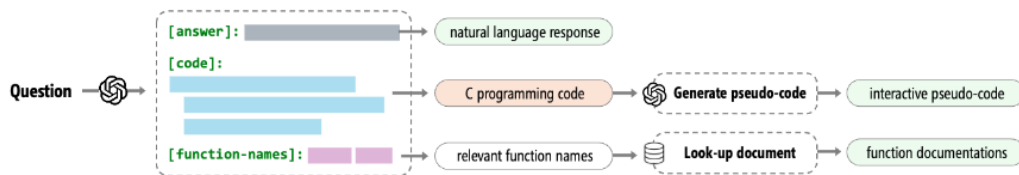


Figura 2.2: Flujo de procesamiento de CodeAid desde la consulta del alumno hasta la generación de pseudocódigo [11].

TreeInstruct

Una de las investigaciones más recientes en este ámbito propone *TreeInstruct*, un *framework* de cuestionamiento guiado por árboles diseñado para el *debugging* educativo [2]. A diferencia de las herramientas comerciales anteriores, este modelo se basa en dos agentes que trabajan de forma coordinada. Por un lado hay un Instructor que diseña la secuencia de preguntas y por otro lado un Verificador que analiza si el estudiante realmente está comprendiendo cada paso en tiempo real.

Para lograr esto el *framework* utiliza variables de estado que funcionan como indicadores del nivel de conocimiento alcanzado en cada momento. Estas variables representan hitos específicos como entender el origen de un fallo o ser capaz de proponer una corrección lógica. Al principio de la sesión estas variables se inicializan en falso y según las respuestas del alumno, el agente Verificador va actualizando los indicadores para que el agente Instructor pueda seleccionar las preguntas más adecuadas dentro del árbol de decisión y asegurar así un aprendizaje progresivo. Tal como se detalla en la figura 2.3, esta estructura garantiza que el progreso dependa del dominio real de los conceptos por parte del

alumno para fomentar un descubrimiento del error basado en su propio razonamiento lógico [2].

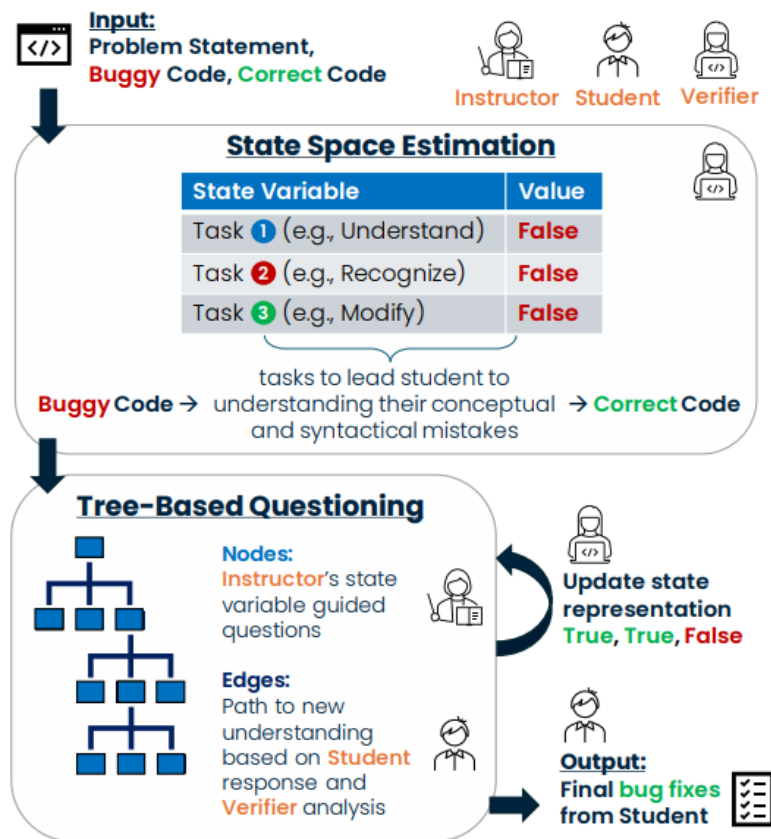


Figura 2.3: Esquema de la arquitectura de TreeInstruct, mostrando la estructura jerárquica de árbol y el flujo de cuestionamiento. [2]

2.2.3. Ingeniería de Prompts en educación

Lograr que un modelo de lenguaje convencional funcione como un tutor socrático que sepa limitarse depende de un diseño de prompts muy cuidado. No basta con darle un papel a la IA sino que hay que estructurar sus instrucciones para que use métodos como la dialéctica o la mayéutica [6].

El principal obstáculo para conseguirlo es que los modelos actuales están diseñados por defecto para actuar como asistentes que ofrecen respuestas directas y rápidas a cualquier consulta. Para mitigar esta tendencia se hace indispensable el uso de un *system prompt* robusto que defina unas reglas sobre cómo debe interactuar el sistema con el estudiante en todo momento. Como se ha podido observar en los casos de éxito analizados anteriormente, existen otras técnicas fundamentales como el *few-shot prompting* que permiten entrenar al modelo mediante ejemplos de conversaciones ideales donde la IA ofrece pistas sutiles en lugar de soluciones completas. Al combinar estas estrategias de diseño se logra que la herramienta mantenga un tono pedagógico constante y se garantiza que el alumno sea quien resuelva el problema mediante su propio razonamiento lógico [2].

2.3. Modelos de Lenguaje Extenso (LLMs) y Despliegue Local

La integración de la Inteligencia Artificial en entornos educativos no es solo una cuestión de pedagogía sino también de infraestructura. A la hora de integrar un modelo de lenguaje como motor cognitivo surge el dilema de si es mejor alquilar ese servicio a empresas externas o si resulta más conveniente hospedar el modelo en servidores propios para mantener el control absoluto sobre el sistema.

2.3.1. APIs en la nube vs. Inferencia Local

Las soluciones basadas en APIs comerciales como las de OpenAI o Anthropic ofrecen una gran potencia con una inversión inicial casi nula, pero introducen riesgos importantes en lo que respecta a la privacidad y los costes a largo plazo. Uno de los principales inconvenientes es que el envío de datos sensibles de los alumnos a servidores externos puede comprometer la seguridad de la información y dificultar el cumplimiento de normativas estrictas como el GDPR [12].

Las APIs funcionan mediante el pago por tokens y esto suele disparar los costes operativos de forma impredecible cuando hay muchos usuarios conectados simultáneamente, lo cual resulta inasumible en entornos educativos con presupuestos limitados. Aunque montar un servidor propio requiere comprar hardware de antemano, esta opción permite un equilibrio financiero donde los costes iniciales serán significativamente más altos, pero predecibles. Investigaciones recientes sitúan el umbral de rentabilidad o *break-even point* en un volumen aproximado de entre 1,5 y 2 millones de tokens diarios ya que a partir de ese nivel de uso la infraestructura propia compensa el gasto acumulado en cuotas de suscripción externa [13]. Alcanzar estas cifras no es raro en el ámbito académico, aún sin un uso intensivo. Esta estrategia permite rentabilizar la inversión inicial y al mismo tiempo asegura la soberanía tecnológica de la institución al evitar la dependencia de proveedores externos que puedan modificar de manera unilateral sus términos de uso o condiciones de servicio.

Si bien es cierto que igualar la capacidad de razonamiento de un modelo comercial masivo mediante una infraestructura local es una tarea compleja un tutor socrático orientado a estudiantes principiantes de ingeniería informática no requiere tal nivel de sofisticación. En este escenario el diseño de la plataforma debe priorizar la protección de la privacidad y el control de los datos del alumnado por encima de una potencia de procesamiento que no llegaría a aprovecharse en su totalidad.

2.3.2. Ecosistema y clasificación de modelos de lenguaje de código abierto

La oferta de modelos de lenguaje de código abierto ha crecido de manera exponencial permitiendo elegir entre diferentes arquitecturas según las necesidades de cada proyecto. Estas inteligencias se distribuyen habitualmente en versiones que varían según su número de parámetros, que son las conexiones internas que determinan la capacidad de razonamiento del sistema. Los modelos suelen etiquetarse con cifras como 1B, 7B o 70B para indicar los miles de millones de parámetros que contienen y esta escala influye directamente tanto en la precisión de las respuestas como en la exigencia de VRAM necesaria para su ejecución local.

En el panorama actual de los Grandes Modelos de Lenguaje (LLM), se distinguen principalmente dos categorías según su metodología de respuesta, los modelos de instrucción (Instruct) y los modelos de razonamiento (Reasoning). Los primeros están optimizados para generar respuestas directas a partir de las instrucciones del usuario, priorizando la concisión y el seguimiento de reglas de formato. Por el contrario, los modelos de razonamiento ejecutan una fase previa de computación interna conocida como Chain of Thought (CoT) [14]. Durante esta etapa, el modelo genera un razonamiento intermedio, frecuentemente encapsulado entre etiquetas como `<think></think>`, que sirve como base lógica para construir la respuesta final. Esta técnica permite al modelo descomponer problemas complejos en pasos manejables, mejorando significativamente su rendimiento en tareas lógicas, matemáticas y de programación.

En el escenario tecnológico actual, existen diversas familias de modelos de código abierto desarrollados por empresas como Meta, Google o Alibaba, cada una con distintos enfoques y cantidad de parámetros. Para cuantificar el rendimiento de estos modelos en el ámbito académico y técnico, se utilizan diversos indicadores estandarizados o benchmarks. En esta comparativa 2.1 se han considerado cuatro métricas clave:

IFEval [15]: Evalúa la adherencia a restricciones instruccionales y de formato, garantizando el cumplimiento de protocolos pedagógicos específicos.

LiveCodeBench (LCB v6) [16]: Mide la capacidad de resolución de problemas técnicos y generación de código, validando su solvencia en entornos de ingeniería.

GPQA [17]: Analiza el conocimiento experto en dominios científicos complejos mediante preguntas de nivel de doctorado resistentes a búsquedas convencionales en la web.

AIME [18]: Evalúa el razonamiento lógico-matemático avanzado, mediante problemas que requieren cadenas de pensamiento complejas. Esta métrica indica la fluidez del modelo y su capacidad de deducción.

Como es de esperar, los modelos de mayor tamaño tienden a ofrecer un rendimiento superior

Modelo	Tipo	Tamaño	Contexto	IFEval	LCB v6	GPQA	AIME
Qwen 3.6 35B	Reasoning	24.0 GB	256k	N/A	80.4	86.0	92.7
Ministral 3 14B	Reasoning	24.0 GB	256k	N/A	64.6	N/A	85.0
Gemma 4 31B	Reasoning	20.0 GB	256k	N/A	80.0	84.3	89.2
Qwen 3.6 27B	Reasoning	17.0 GB	256k	95.0	83.9	87.8	94.1
Phi-4 Rsn 14B	Reasoning	11.0 GB	32k	83.4	53.8	67.1	62.9
Gemma 4 e4B	Reasoning	9.6 GB	128k	N/A	52.0	58.6	42.5
Phi-4 14B	Instruct	9.1 GB	16k	62.3	26.9	54.7	12.9
Gemma 4 e2B	Reasoning	7.2 GB	128k	N/A	44.0	43.4	37.5
Qwen 3.5 9B	Reasoning	6.6 GB	256k	91.5	65.6	81.7	N/A
Ministral 3 8B	Reasoning	6.0 GB	256k	N/A	61.6	N/A	78.7
DeepSeek-R1 8B	Reasoning	5.2 GB	128k	37.8	39.6	49.0	43.3
Qwen 3.5 4B	Reasoning	3.4 GB	256k	89.8	55.8	76.2	N/A
Gemma 3 4B	Instruct	3.3 GB	128k	90.2	23.0	30.8	N/A
Phi-4 Mini Rsn 3.8B	Reasoning	3.2 GB	128k	N/A	N/A	52.0	57.5
Ministral 3 3B	Reasoning	3.0 GB	256k	N/A	54.8	N/A	72.1
Phi-4 Mini 3.8B	Instruct	2.5 GB	128k	N/A	N/A	25.2	10.0
Llama 3.2 3B	Instruct	2.0 GB	128k	73.9	N/A	32.8	N/A

Tabla 2.1: Comparativa de arquitecturas de modelos de lenguaje verificada mediante la biblioteca de Ollama (abril 2026). Los datos relativos al tamaño en disco y la ventana de contexto han sido extraídos directamente de las especificaciones de despliegue local de Ollama. Por su parte, las métricas de rendimiento (IFEval, LiveCodeBench v6, GPQA Diamond y AIME) provienen de los informes técnicos oficiales de sus respectivos desarrolladores y de la documentación técnica disponible en Hugging Face [19–27]. Los valores marcados como N/A indican la ausencia de datos oficiales o pruebas independientes bajo condiciones de cuantización idénticas en las fuentes consultadas.

en los benchmarks, sin embargo su despliegue local resulta más costoso y complejo. Por otro lado, los modelos más ligeros presentan un rendimiento menor, pero permiten una implementación local más sencilla. Destaca la notable diferencia entre los modelos de instrucción y los de razonamiento, obteniendo estos últimos mejores resultados a costa de un mayor tiempo de respuesta durante el proceso de inferencia.

2.3.3. Optimización de modelos ligeros

El principal limitador técnico para el despliegue de estos sistemas en servidores propios es el consumo de la memoria de video o VRAM de la tarjeta gráfica. Cada parámetro del modelo debe cargarse en este espacio para que la generación de texto sea rápida y eficiente. Por defecto los modelos se entrenan con una precisión de 16 bits lo cual implica que una arquitectura de pocos parámetros puede requerir una cantidad de memoria inasumible para hardware de rango medio.

Para superar las limitaciones de hardware, se han implementado diversas estrategias de optimización que actúan sobre la arquitectura y la representación de los datos. La optimización técnica comienza con los formatos de precisión que determinan cuántos bits utiliza el hardware para representar cada decimal de la arquitectura. Al pasar de estándares de dieciséis bits a otros más reducidos como el FP8 se ahorra espacio de forma nativa sin perder apenas precisión [28]. Este método se refuerza con la cuantización de parámetros la cual actúa como una compresión matemática mucho más agresiva mediante la discretización de los pesos y su mapeo sistemático hacia un dominio numérico finito [29]. Mientras los formatos de precisión reducen el tamaño estructural del contenedor, la cuantización re-define el contenido para garantizar que un modelo de gran escala quepa en la VRAM de una tarjeta gráfica doméstica.

Para gestionar las limitaciones físicas del hardware empleamos el offloading como una técnica logística que distribuye las capas del modelo entre la memoria de vídeo y la memoria RAM del sistema, permitiendo cargar un modelo que demanda más VRAM de la que dispone la tarjeta gráfica. Toda esta infraestructura se beneficia de la arquitectura Mixture of Experts o MoE donde el sistema solo activa una pequeña fracción de sus parámetros para responder a cada pregunta concreta [30]. Por ejemplo, los modelos Gemma 4 e4B, Gemma 4 e2B y Qwen 3.6 35B de la tabla 2.1 utilizan esta arquitectura para agilizar la latencia de respuesta preservando la base de conocimiento.

2.4. Seguridad y aislamiento en la ejecución de código

El diseño de una plataforma interactiva de programación requiere integrar un entorno capaz de compilar y ejecutar el código fuente de los estudiantes para ofrecer una experiencia pedagógica interactiva basada en la retroalimentación inmediata. Esta necesidad funcional implica habilitar la ejecución de

código arbitrario como un servicio el cual expone inevitablemente al servidor anfitrión a múltiples vectores de amenaza como la escalada de privilegios para acceder al sistema de archivos, la exfiltración de credenciales sensibles mediante conexiones de red salientes y los ataques de denegación de servicio (DoS) provocados por bucles infinitos o la creación exponencial de procesos mediante *fork bombs* destinados a agotar los recursos del sistema operativo [31]. Para mitigar estos riesgos, es necesario implementar un confinamiento estricto de procesos mediante técnicas de *sandboxing* que garanticen que el código no confiable funcione en un entorno estanco y sin privilegios de administración [32].

Frente a estos riesgos las plataformas web modernas descartan las máquinas virtuales tradicionales debido a su alta latencia y exploran alternativas como la ejecución en el cliente mediante **Pyodide** la cual utiliza *WebAssembly* para procesar el código directamente en el navegador del estudiante. Aunque este enfoque garantiza una seguridad absoluta para el servidor al delegar el cómputo íntegramente al hardware del usuario, presenta desventajas como la elevada carga inicial de librerías pesadas y la dependencia de los recursos locales del dispositivo del alumno lo que dificulta una experiencia de tutoría consistente. Por otro lado están los motores de ejecución efímeros basados en la virtualización ligera del núcleo de Linux donde destaca **Piston** al orquestar trabajos aislados (*jobs*) mediante el uso coordinado de *namespaces* y *cgroups* para seccionar la visibilidad de red y limitar el consumo de recursos físicos de forma simultánea [33]. Este software protege la infraestructura al ejecutar cada envío en directorios temporales montados sobre la memoria RAM y bajo identidades de usuario huérfanas sin privilegios administrativos garantizando que cualquier proceso sospechoso sea interceptado por un supervisor temporal que termina la ejecución automáticamente antes de comprometer la integridad del sistema global [34]. No obstante es fundamental reconocer que la ejecución de código arbitrario siempre conlleva un peligro por lo que estas capas de blindaje se plantean como una estrategia de defensa en profundidad orientada a mitigar los riesgos conocidos sin ignorar la posibilidad teórica de vulnerabilidades imprevistas en el aislamiento del núcleo.

2.5. Arquitecturas web para entornos interactivos

El desarrollo de plataformas educativas interactivas, como *SocratiCode*, exige una infraestructura de *software* capaz de soportar una alta carga de interactividad concurrente. Al estar formado por herramientas de gran complejidad, como un editor de código avanzado, una terminal de ejecución sincronizada y un chat asistido por Inteligencia Artificial, requiere un flujo de datos continuo.

2.5.1. Paradigma de Aplicación de Página Única (SPA)

Históricamente, el desarrollo de plataformas web se ha fundamentado en el modelo de *Multi-Page Application* (MPA), donde cada interacción del usuario requiere una petición al servidor y la recarga completa del documento HTML, resultando ineficaces en términos de latencia y persistencia de estado.

No obstante, la evolución hacia herramientas de alta interactividad y manipulación de datos en tiempo real ha consolidado a las **Single-Page Applications** (SPA) como el estándar de la industria, adoptado por referentes como *VS Code Web*, *GitHub Codespaces* o las interfaces de *ChatGPT*. Con esta arquitectura, el navegador descarga inicialmente un «caparazón» o *shell* de la aplicación, a partir de ese momento, la comunicación con el servidor se limita exclusivamente al intercambio de datos, generalmente en formato JSON, permitiendo que la página nunca se recargue globalmente, sino que se transforme de manera dinámica. Esta arquitectura hace que la experiencia de usuario (UX) sea similar a la de las aplicaciones de escritorio, minimizando la latencia percibida y eliminando las interrupciones visuales que entorpecen el flujo de trabajo [35].

2.5.2. Protocolos de comunicación en tiempo real

La integración de interfaces de chat asistidas por modelos de lenguaje en aplicaciones web introduce desafíos críticos relacionados con la latencia computacional. En los sistemas de mensajería convencionales, se emplean peticiones HTTP bajo el ciclo tradicional de solicitud y respuesta, en este modelo el cliente debe esperar a que el servidor genere la totalidad del mensaje antes de recibir cualquier dato. Sin embargo, debido a la naturaleza autorregresiva de los modelos de lenguaje de gran escala, la inferencia completa de una respuesta extensa puede demorar varios segundos, lo que resultaría en una mala experiencia de usuario debido a los tiempos de espera.

La solución adoptada por los líderes de la industria, tales como OpenAI (ChatGPT) o Anthropic (Claude) [36], consiste en la transmisión progresiva de datos o *streaming*. Este mecanismo permite visualizar la respuesta fragmento a fragmento (*token a token*) conforme el modelo la genera, optimizando drásticamente la latencia percibida. Técnicamente, este flujo de datos se implementa principalmente a través de **WebSockets** o de **Server-Sent Events (SSE)**. Mientras que el primer protocolo es más complejo al establecer una comunicación bidireccional y *full-duplex* que requiere un cambio de protocolo (*protocol upgrade*) y una gestión de estado más costosa en el servidor, el segundo destaca por su ligereza y su integración nativa con el estándar HTTP [37].

En consecuencia, *Server-Sent Events* se ha consolidado como el estandar en la arquitectura de servicios de IA generativa [38], ya que permite a las grandes plataformas mantener una infraestructura escalable y eficiente para flujos de datos unidireccionales, garantizando la fluidez de la interfaz sin la sobrecarga técnica de los canales bidireccionales.

2.6. Conclusiones del Estado del Arte

En conclusión, el análisis del presente capítulo evidencia que, si bien la Inteligencia Artificial generativa ha alcanzado un grado de madurez significativo en la asistencia al desarrollo de *software*, aún

persisten retos críticos relacionados con el riesgo de facilismo algorítmico en la educación, la elevada tasa de abandono en los cursos introductorios de programación (CS1) y la dependencia de infraestructuras comerciales que comprometen la soberanía de los datos. Este panorama justifica la necesidad de diseñar una solución como *SocratiCode*, que combine la metodología socrática de andamiaje cognitivo con una arquitectura técnica basada en modelos de lenguaje locales y protocolos de transmisión en tiempo real, fomentando así un enfoque que priorice el descubrimiento guiado y la autonomía del estudiante en la construcción de su propio pensamiento computacional.

ANÁLISIS Y DISEÑO

3.1. Metodología

3.2. Descripción del sistema - SocratiCode

3.3. Requisitos funcionales y no funcionales

3.3.1. Requisitos funcionales

3.3.2. Requisitos no funcionales

3.4. Casos de uso

3.5. Diagrama de modelo de datos (Clases/Entidad-Relación)

3.6. Diagrama de componentes y arquitectura

3.6.1. Diagrama de arquitectura

3.6.2. Diagramas de secuencia de las funcionalidades principales

3.7. Conclusiones

DESARROLLO E IMPLEMENTACIÓN

4.1. Entorno de programación

4.2. Tecnologías y librerías

4.2.1. Framework Backend (Django / Python)

4.2.2. Framework Frontend (Vue 3 / Vite)

4.2.3. Motor de Inteligencia Artificial (Ollama / Llama 3.2)

4.2.4. Motor de compilación y ejecución (Piston / Docker)

4.2.5. Control de versiones y despliegue (Git / GitHub)

4.3. Implementación

4.3.1. Gestión de la lógica socrática (Prompt Engineering)

4.3.2. Implementación del flujo de datos en tiempo real (SSE)

4.3.3. Integración y aislamiento del motor de ejecución (Docker/Piston)

4.4. Interfaz

4.4.1. Navegación y vista principal del tutor

4.4.2. Personalización y editor de código

4.5. Verificación y Pruebas del sistema

4.6. Conclusiones

PRUEBAS

- 5.1. Pruebas de seguridad e integridad (Pentesting del Sandbox)**
- 5.2. Encuesta de experiencia de usuario y validación pedagógica**

CONCLUSIONES Y TRABAJO FUTURO

6.1. Conclusiones

6.2. Trabajo futuro

BIBLIOGRAFÍA

- [1] Z. Alshaikh, L. Tamang, and V. Rus, “A socratic tutor for source code comprehension,” in *International Conference on Artificial Intelligence in Education (AIED)*, pp. 15–19, Springer, 2020.
- [2] P. Kargupta, I. Agarwal, D. H. Tur, and J. Han, “Instruct, not assist: Llm-based multi-turn planning and hierarchical questioning for socratic code debugging,” in *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 9443–9458, 2024.
- [3] E. Al-Hossami, R. Bunescu, J. Smith, and R. Teehan, “Can language models employ the socratic method? experiments with code debugging,” in *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1*, pp. 53–59, 2024.
- [4] V. Aleven and K. R. Koedinger, “An effective metacognitive strategy: Learning by doing and explaining with a computer-based cognitive tutor,” *Cognitive Science*, vol. 26, no. 2, pp. 147–179, 2002.
- [5] D. Wood, J. S. Bruner, and G. Ross, “The role of tutoring in problem solving,” *Journal of Child Psychology and Psychiatry*, vol. 17, no. 2, pp. 89–100, 1976.
- [6] E. Y. Chang, “Prompting large language models with the socratic method,” in *IEEE Computing and Communication Workshop and Conference (CCWC)*, 2023.
- [7] B. Chen, C. M. Lewis, M. West, and C. Zilles, “Plagiarism in the age of generative ai: Cheating method change and learning loss in an intro to cs course,” in *Proceedings of the Eleventh ACM Conference on Learning at Scale*, pp. 75–85, 2024.
- [8] J. Prather *et al.*, “Widening the gap: Genai tools can compound the metacognitive difficulties novices face when learning to program,” in *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1*, pp. 276–282, 2024.
- [9] D. J. Malan, “Cs50x 2026 - artificial intelligence.” Vídeo en YouTube, 2026. Accedido el 24 de abril de 2026.
- [10] R. Liu, C. Zenke, C. Liu, and D. J. Malan, “Teaching cs50 with ai: Leveraging generative artificial intelligence in computer science education,” in *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1*, 2024.
- [11] M. Kazemitabaar, R. Ye, X. Wang, A. Z. Henley, P. Denny, M. Craig, and T. Grossman, “Codeaid: Evaluating a classroom deployment of an llm-based programming assistant that balances student and educator needs,” in *Proceedings of the CHI Conference on Human Factors in Computing Systems*, pp. 1–17, 2024.
- [12] A. Wada *et al.*, “Local llm deployment for enhanced privacy,” *arXiv preprint*, 2025.
- [13] G. Pan and H. Wang, “A cost-benefit analysis of on-premise large language model deployment: Breaking even with commercial llm services,” in *2025 IEEE International Conference on Big Data (BigData)*, 2025.

- [14] J. Wei, X. Wang, D. Schuurmans, M. Bosma, b. ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems* (S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, eds.), vol. 35, pp. 24824–24837, Curran Associates, Inc., 2022.
- [15] J. Zhou *et al.*, “Instruction-following evaluation through constraint checklists,” *arXiv preprint arXiv:2311.07911*, 2023.
- [16] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica, “Livecodebench: Holistic and contamination free evaluation of large language models for code,” 2024.
- [17] D. Rein *et al.*, “Gpqa: A graduate-level google-proof q&a benchmark,” *arXiv preprint arXiv:2311.12022*, 2023.
- [18] Mathematical Association of America, “American invitational mathematics examination (aime) standards,” 2026.
- [19] Meta AI, “Llama 3.2: Vision, edge, and mobile devices,” *Meta AI Blog*, 2024.
- [20] G. Team, “Gemma 3,” 2025.
- [21] DeepSeek-AI, “Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning,” 2025.
- [22] Qwen Team, “Qwen3.5: Towards native multimodal agents,” February 2026.
- [23] Qwen Team, “Qwen3.6-27B: Flagship-level coding in a 27B dense model,” April 2026.
- [24] Qwen Team, “Qwen3.6-35B-A3B: Agentic coding power, now open to all,” April 2026.
- [25] Microsoft Research, “Phi-4 technical report: Reasoning and instruction following at scale,” tech. rep., Microsoft, 2025.
- [26] Microsoft Research, “Phi-4-mini technical report,” tech. rep., Microsoft, 2025.
- [27] Hugging Face, “Hugging face model hub: Central repository for pre-trained models and benchmarks,” 2026. Used for technical specifications, model cards, and benchmark verification. Accessed: 2026-04-28.
- [28] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, *et al.*, “Mixed precision training,” *arXiv preprint arXiv:1710.03740*, 2018. Explica la base técnica de los formatos FP16, BF16 y la reducción de precisión.
- [29] A. Yadav and R. C. Bhargavi, “Optimizing llms using quantization for mobile execution,” in *Proceedings of ICT4SD 2025*, Springer, 2025.
- [30] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” *arXiv preprint arXiv:1701.06538*, 2017. Referencia fundamental sobre modelos MoE y parámetros activos.
- [31] Y. Sun, S. Nanda, and T. Jaeger, “Security namespace: Making linux security frameworks available to containers,” in *27th USENIX Security Symposium (USENIX Security 18)*, pp. 1423–1439, 2018.
- [32] Z. Wan, D. Lo, X. Xia, and L. Cai, “Practical and effective sandboxing for linux containers,” *Empirical Software Engineering*, vol. 24, no. 6, pp. 4034–4070, 2019.

- [33] R. Rosen, "Resource management: Linux kernel namespaces and cgroups," *Haifux*, vol. 186, 2013.
- [34] Engineer Man, "Piston: A high performance general purpose code execution engine," 2025.
- [35] M. Mikowski and J. Powell, *Single page web applications: JavaScript end-to-end*. Manning Publications, 2013.
- [36] OpenAI, "Api reference: Streaming responses," 2024. Accedido: 30-04-2026.
- [37] M. MACKOVIČ, "Comparative analysis of websockets and server-sent events: Performance, use cases, and best practices," Análisis técnico de los protocolos WebSockets y Server-Sent Events (SSE), evaluando su rendimiento, casos de uso y mejores prácticas para aplicaciones web modernas.
- [38] P. Tech, "Server-sent events for llms: The backbone of ai applications." <https://procedure.tech/blogs/sse-for-llms/>, 2025. Reporte técnico de ingeniería arquitectónica sobre por qué SSE supera a WebSockets y gRPC en la entrega optimizada y escalable de tokens generados por LLMs.

APÉNDICES

SPRINTS DESARROLLADOS

DIAGRAMAS Y CASOS DE USO

NAVEGACIÓN E INTERFACES EXTRA



RESULTADOS DE ENCUESTA Y AUDITORÍA DE SEGURIDAD



Universidad Autónoma
de Madrid